Claude Code Chat History – Lab 4

Extracted conversational messages: 39

User

I am working on High-Low Card Predictor game written in python where the player predicts whether the next card drawn from a standard 52-card deck will have a higher or lower rank than the current card. Players earn points for correct predictions, with the game tracking their score and remaining cards in the deck.
The game currently has a comparison bug because card ranks can be evaluated alphabetically instead of by their actual values.
Expected: 2 < 3 < ... < 10 < J < Q < K < A
Inspect the current code files and fix the comparison at the appropriate location, and test the edge cases involving 9, 10, J, Q, K .

User

[Tool result]
(Bash completed with no output)

User

[Tool result]
/Users/sonia/Software-Engineering-Lab
./README.md
./.gitignore
./main.py
./game/deck.py
./game/game_engine.py
./game/card.py
=== ./main.py
```
     1  import pygame
     2  from game.game_engine import GameEngine
     3
     4  WIDTH, HEIGHT = 650, 440
     5  FPS = 60
     6
     7
     8  def main():
     9      pygame.init()
    10      screen = pygame.display.set_mode((WIDTH, HEIGHT))
    11      pygame.display.set_caption("High-Low Card Predictor -
Pygame Edition")
    12      clock = pygame.time.Clock()
```

```

    engine = GameEngine(WIDTH, HEIGHT)

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.update()
        engine.render(screen)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()=== ./game/deck.py
import random
from game.card import Card


class Deck:

    SUITS = ["Hearts", "Diamonds", "Clubs", "Spades"]
    RANKS = [
        ("2", 2), ("3", 3), ("4", 4), ("5", 5), ("6", 6),
        ("7", 7), ("8", 8), ("9", 9), ("10", 10),
        ("J", 11), ("Q", 12), ("K", 13), ("A", 14)
    ]

    def __init__(self):
        self.cards = []
        self.reset()

    def reset(self):
        self.cards = [Card(r[0], s, r[1]) for s in
self.SUITS for r in self.RANKS]
        random.shuffle(self.cards)

    def draw(self):
        if not self.cards:
            self.reset()
        return self.cards.pop()

    @property
    def remaining(self):
        return len(self.cards)=== ./game/game_engine.py
import pygame
from game.deck import Deck

```

```
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.deck = Deck()

        self.current_card = self.deck.draw()
        self.next_card = None
        self.score = 0
        self.status_msg = "Will the next card be HIGHER or LOWER?"
        self.status_color = (220, 220, 220)

        btn_w, btn_h = 140, 48
        self.btn_higher = pygame.Rect(width // 2 - btn_w - 20, height - 90, btn_w, btn_h)
        self.btn_lower = pygame.Rect(width // 2 + 20, height - 90, btn_w, btn_h)

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_medium = pygame.font.SysFont(None, 30)
        self.font_small = pygame.font.SysFont(None, 24)

    def evaluate_guess(self, guess):
        """Draws next card and evaluates prediction."""
        self.next_card = self.deck.draw()

        if guess == "HIGHER":
            correct = self.next_card.rank_str > self.current_card.rank_str
        else:
            correct = self.next_card.rank_str < self.current_card.rank_str

        if correct:
            self.score += 1
            self.status_msg = f"CORRECT! {self.next_card.rank_str} vs {self.current_card.rank_str}"
            self.status_color = (80, 220, 80)
        else:
            self.score = max(0, self.score - 1)
            self.status_msg = f"WRONG! {self.next_card.rank_str} vs {self.current_card.rank_str}"
            self.status_color = (235, 75, 75)

        self.current_card = self.next_card

    def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.btn_higher.collidepoint(event.pos):
                self.evaluate_guess("HIGHER")
            elif self.btn_lower.collidepoint(event.pos):
                self.evaluate_guess("LOWER")
```

```
    50
    51      def update(self):
    52          pass
    53
    54      def render(self, screen):
    55          screen.fill((25, 80, 45))
    56
    57          title_surf = self.font_title.render("High-Low Card
Predictor", True, (245, 245, 245))
    58          screen.blit(title_surf, (self.width // 2 -
title_surf.get_width() // 2, 25))
    59
    60          score_surf = self.font_medium.render(f"Score:
{self.score}", True, (255, 220, 80))
    61          screen.blit(score_surf, (30, 30))
    62
    63          rem_surf = self.font_small.render(f"Deck:
{self.deck.remaining} left", True, (210, 210, 210))
    64          screen.blit(rem_surf, (self.width -
rem_surf.get_width() - 30, 35))
    65
    66          card_w, card_h = 130, 180
    67          self.current_card.render(screen, self.width // 2 -
card_w // 2, 100, card_w, card_h)
    68
    69          status_surf =
self.font_small.render(self.status_msg, True, self.status_color)
    70          screen.blit(status_surf, (self.width // 2 -
status_surf.get_width() // 2, 310))
    71
    72          pygame.draw.rect(screen, (40, 140, 60),
self.btn_higher, border_radius=8)
    73          pygame.draw.rect(screen, (220, 220, 220),
self.btn_higher, width=2, border_radius=8)
    74          high_surf = self.font_medium.render("HIGHER", True,
(255, 255, 255))
    75          screen.blit(
    76              high_surf,
    77              (self.btn_higher.centerx -
high_surf.get_width() // 2, self.btn_higher.centery -
high_surf.get_height() // 2),
    78          )
    79
    80          pygame.draw.rect(screen, (170, 50, 50),
self.btn_lower, border_radius=8)
    81          pygame.draw.rect(screen, (220, 220, 220),
self.btn_lower, width=2, border_radius=8)
    82          low_surf = self.font_medium.render("LOWER", True,
(255, 255, 255))
    83          screen.blit(
    84              low_surf,
    85              (self.btn_lower.centerx -
low_surf.get_width() // 2, self.btn_lower.centery -
low_surf.get_height() // 2),
```

```
    86          )
=== ./game/card.py
     1  import pygame
     2
     3
     4  class Card:
     5
     6      def __init__(self, rank_str, suit_str, numeric_rank):
     7          self.rank_str = rank_str
     8          self.suit_str = suit_str
     9          self.numeric_rank = numeric_rank
    10
    11          self.symbol = {"Hearts": "♥", "Diamonds": "♦",
"Clubs": "♣", "Spades": "♠"}.get(suit_str, "")
    12          self.is_red = suit_str in ("Hearts", "Diamonds")
    13          self.color = (220, 40, 40) if self.is_red else (30,
30, 30)
    14
    15      def render(self, surface, x, y, width=130, height=180):
    16          card_rect = pygame.Rect(x, y, width, height)
    17          pygame.draw.rect(surface, (255, 255, 255),
card_rect, border_radius=10)
    18          pygame.draw.rect(surface, (50, 50, 50), card_rect,
width=3, border_radius=10)
    19
    20          font_rank = pygame.font.SysFont(None, 36)
    21          font_symbol = pygame.font.SysFont(None, 64)
    22
    23          rank_surf = font_rank.render(self.rank_str, True,
self.color)
    24          surface.blit(rank_surf, (x + 10, y + 8))
    25
    26          symbol_surf = font_symbol.render(self.symbol, True,
self.color)
    27          surface.blit(
    28              symbol_surf,
    29              (x + width // 2 - symbol_surf.get_width() // 2,
y + height // 2 - symbol_surf.get_height() // 2),
    30          )
    31
    32          surface.blit(rank_surf, (x + width -
rank_surf.get_width() - 10, y + height - rank_surf.get_height() -
8))
```

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully.

User

[Tool result]
test_edge_cases_9_10_face_cards
(tests.test_comparison.TestRankComparison.test_edge_cases_9_10_face_
cards) ... ok
test_equal_rank_is_not_correct
(tests.test_comparison.TestRankComparison.test_equal_rank_is_not_cor
rect) ... ok
test_full_rank_order
(tests.test_comparison.TestRankComparison.test_full_rank_order) ...
ok

--
Ran 3 tests in 0.799s

OK
pygame 2.6.1 (SDL 2.28.4, Python 3.12.15)
Hello from the pygame community. https://www.pygame.org/
contribute.html
--- before fix:
error: pathspec ':(prefix:23)Lab4/55_card_predictor/game/
game_engine.py' did not match any file(s) known to git

Assistant

All tests pass with the fix. I couldn't stash to check the old code
because these files aren't tracked by git at this path. Instead,
I'll run the tests against a temporary copy of the original code:

User

[Tool result]
FAIL: test_edge_cases_9_10_face_cards
(tests.test_comparison.TestRankComparison.test_edge_cases_9_10_face_
cards) (low='9', high='10')
AssertionError: False is not true
FAIL: test_edge_cases_9_10_face_cards
(tests.test_comparison.TestRankComparison.test_edge_cases_9_10_face_
cards) (low='Q', high='K')
AssertionError: False is not true
FAIL: test_edge_cases_9_10_face_cards
(tests.test_comparison.TestRankComparison.test_edge_cases_9_10_face_
cards) (low='K', high='A')
AssertionError: False is not true
FAIL: test_edge_cases_9_10_face_cards
(tests.test_comparison.TestRankComparison.test_edge_cases_9_10_face_

```
cards) (low='Q', high='A')
AssertionError: False is not true
FAIL: test_full_rank_order
(tests.test_comparison.TestRankComparison.test_full_rank_order)
(low='9', high='10')
AssertionError: False is not true
FAIL: test_full_rank_order
(tests.test_comparison.TestRankComparison.test_full_rank_order)
(low='Q', high='K')
AssertionError: False is not true
Shell cwd was reset to /Users/sonia/Software-Engineering-Lab/
Lab4/55_card_predictor
```

Assistant

I fixed the comparison bug, and the new tests pass, including the 9, 10, J, Q, K edge cases.

Where the bug was: [game_engine.py:28-31](game/game_engine.py#L28-L31). `evaluate_guess` compared the cards' rank text (`rank_str`, like `"9"` or `"K"`) instead of their numbers. Text is compared letter by letter, so the game got several cases wrong:
- `"10" < "9"`, because `'1'` comes before `'9'`
- `"K" < "Q"` and `"A" < "K"`, because that's alphabetical order

The fix: both checks now use `numeric_rank`, the number from 2 to 14 that [deck.py](game/deck.py#L8-L12) already gives each card. That gives the order you expected: 2 < 3 < … < 10 < J < Q < K < A. The rest of the code (deck, card, display) didn't need changes.

Tests: I added tests/test_comparison.py. Each test runs the real `evaluate_guess` with a stand-in deck that always deals a chosen card, and checks the score. They cover:
- every neighbouring pair from 2 up to A, guessing both HIGHER and LOWER
- the edge cases you asked about: 9↔10, 9↔J/Q/K, 10↔J/Q/K/A, J↔Q, J↔K, Q↔K, K↔A, Q↔A
- two cards of the same rank, which count as a wrong guess either way

All 3 tests pass with the fix. Run against a copy of the original code, they fail on 9 vs 10, Q vs K, K vs A and Q vs A, which confirms they catch this bug. To run them:

```
.venv/bin/python -m unittest tests.test_comparison -v
```

User

Now I want to add a new feauture to this game :
Correct guesses should build a winning streak and increase the score reward through escalating multipliers.
- Track consecutive correct guesses.
- Increase the multiplier as the streak grows.
- Reset the streak bonus after an incorrect guess.
- Display the streak or multiplier clearly.

Assistant

I'll keep the scoring rules in a small helper so the escalation is easy to tune and test. Here's the multiplier schedule I'm going with: x1 for streaks 1–2, x2 for 3–4, x3 for 5–6, x4 for 7–8, and x5 (the cap) from 9 onward.

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/

Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
File created successfully at: /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/tests/test_streak.py (file state is current in your context — no need to Read it back)

Assistant

I added the streak multiplier, and all 8 tests pass: the 3 earlier comparison tests and 5 new streak tests. I haven't opened the game window to look at the new streak line, so give it a quick play to check the layout.

How it works

Streak	Multiplier	Points per correct guess
1–2	x1	+1
3–4	x2	+2
5–6	x3	+3
7–8	x4	+4
9+	x5 (cap)	+5

- **Tracking:** the game now keeps a count of correct guesses in a row (`self.streak`). Each correct guess adds 1 to it, and the points for that guess are the multiplier for the new streak length.
- **Escalation:** the multiplier goes up by 1 every 2 correct guesses, up to x5. Two settings at the top of [game_engine.py](game/game_engine.py) control this: `STREAK_STEP` (how many correct guesses per step) and `MAX_MULTIPLIER` (the cap).
- **Reset:** a wrong guess sets the streak back to 0 and the multiplier back to x1. It still costs 1 point, as before, and the score can't go below 0. If you had a streak of 2 or more, the message also says "Streak of N lost!". Two cards of the same rank count as wrong, so they also end the streak.
- **Display:**
  - A new line under the score reads `Streak: N  xM`. It turns orange once the multiplier reaches x2 or more.
  - The message after a correct guess now shows the points earned, e.g. `CORRECT! 7 vs 5  +2 (x2)`.

Tests: the new tests are in tests/test_streak.py. They cover:

- the multiplier schedule in the table
- the score climbing over 5 correct guesses in a row (1 → 2 → 4 → 6 → 9)
- a wrong guess resetting the streak, then the next correct guess scoring x1 again
- two cards of the same rank breaking the streak
- drawing the screen with the streak line, which runs without errors

Run them with:
```
.venv/bin/python -m unittest tests.test_comparison tests.test_streak -v
```
The shorter `python -m unittest discover` doesn't work yet because `tests/` has no `__init__.py` file. Adding an empty one would fix that.

User

Next I want to add another feature to implement tie handling :

When the next card has the same rank as the current card, treat it as a tie (a PUSH) instead of a loss.
- Preserve the player's score.
- Preserve the active winning streak.
- Display a neutral PUSH message.
- Do not award or deduct points for the tie unless the existing specification says otherwise.

Assistant

The tie check goes in `evaluate_guess`, before the higher/lower comparison. Two existing tests assumed a tie counts as a loss, so I'm updating those as well.

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/tests/test_comparison.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/

Lab4/55_card_predictor/tests/test_streak.py has been updated successfully. (file state is current in your context — no need to Read it back)

Assistant

A tie now counts as a PUSH instead of a loss, and all 9 tests pass.

What changed: in [game_engine.py](game/game_engine.py), `evaluate_guess` now checks for a tie first. If the two cards have the same rank, whether you guessed HIGHER or LOWER:
- **Score:** stays the same. Nothing is added or taken away.
- **Streak:** stays the same and so does the multiplier. If you were on a streak of 3 at x2, you're still on 3 at x2, and the next correct guess counts as your 4th in a row.
- **Message:** `PUSH! 5 vs 5  No points won or lost`, in the same light grey as the starting prompt. That keeps it separate from the green CORRECT and red WRONG messages.
- **Next round:** the drawn card becomes the current card, as it does after any other guess.

Tests: two old tests assumed a tie counts as a loss, so I replaced them:
- In tests/test_comparison.py, `test_equal_rank_is_push` checks that a tie on 9, 10, J, Q or K keeps the score at 5 and shows a PUSH message, whichever way you guessed.
- In tests/test_streak.py, `test_push_preserves_score_and_streak` builds a streak of 3, hits two ties in a row, and checks that the streak, score and multiplier stay the same through both. It then checks that the next correct guess carries on from there.
- Also in tests/test_streak.py, `test_push_at_zero_streak` checks that a tie with no streak leaves everything at 0.

The PUSH message is the longest status line so far and is drawn centred under the card. It should fit the 650px window, but I haven't checked it on screen.

User

Next I want to add a UI feature :

Instead of immediately replacing the current card, briefly show the previous and newly drawn cards side by side.
Reveal both cards together.
Add a short pause so the comparison is visible.
Then advance to the next round.

Keep button interaction and score updates working correctly.

User

[Tool result]
```
1       import pygame
2       from game.deck import Deck
3
4       class GameEngine:
5           STREAK_STEP = 2       # correct guesses needed to raise
the multiplier
6           MAX_MULTIPLIER = 5
7
8           def __init__(self, width, height):
9               self.width = width
10              self.height = height
11              self.deck = Deck()
12
13              self.current_card = self.deck.draw()
14              self.next_card = None
15              self.score = 0
16              self.streak = 0
17              self.status_msg = "Will the next card be HIGHER or
LOWER?"
18              self.status_color = (220, 220, 220)
19
20              btn_w, btn_h = 140, 48
21              self.btn_higher = pygame.Rect(width // 2 - btn_w -
20, height - 90, btn_w, btn_h)
22              self.btn_lower = pygame.Rect(width // 2 + 20, height
- 90, btn_w, btn_h)
23
24              self.font_title = pygame.font.SysFont(None, 40)
25              self.font_medium = pygame.font.SysFont(None, 30)
26              self.font_small = pygame.font.SysFont(None, 24)
27
28          @classmethod
29          def multiplier_for(cls, streak):
30              """Multiplier earned by the guess that brings the
streak to `streak`."""
31              if streak <= 0:
32                  return 1
33              return min(1 + (streak - 1) // cls.STREAK_STEP,
cls.MAX_MULTIPLIER)
34
35          @property
36          def multiplier(self):
37              return self.multiplier_for(self.streak)
38
39          def evaluate_guess(self, guess):
40              """Draws next card and evaluates prediction."""
41              self.next_card = self.deck.draw()
```

```
42
43              if self.next_card.numeric_rank ==
self.current_card.numeric_rank:
44                  # Tie: a PUSH leaves score and streak untouched.
45                  self.status_msg = f"PUSH!
{self.next_card.rank_str} vs {self.current_card.rank_str}  No points
won or lost"
46                  self.status_color = (220, 220, 220)
47                  self.current_card = self.next_card
48                  return
49
50              if guess == "HIGHER":
51                  correct = self.next_card.numeric_rank >
self.current_card.numeric_rank
52              else:
53                  correct = self.next_card.numeric_rank <
self.current_card.numeric_rank
54              if correct:
55                  self.streak += 1
56                  points = self.multiplier
57                  self.score += points
58                  self.status_msg = (
59                      f"CORRECT! {self.next_card.rank_str} vs
{self.current_card.rank_str}"
60                      f"  +{points} (x{points})"
61                  )
62                  self.status_color = (80, 220, 80)
63              else:
64                  lost_streak = self.streak
65                  self.streak = 0
66                  self.score = max(0, self.score - 1)
67                  self.status_msg = f"WRONG!
{self.next_card.rank_str} vs {self.current_card.rank_str}"
68                  if lost_streak >= 2:
69                      self.status_msg += f"  Streak of
{lost_streak} lost!"
70                  self.status_color = (235, 75, 75)
71
72              self.current_card = self.next_card
73
74          def handle_event(self, event):
75              if event.type == pygame.MOUSEBUTTONDOWN and
event.button == 1:
76                  if self.btn_higher.collidepoint(event.pos):
77                      self.evaluate_guess("HIGHER")
78                  elif self.btn_lower.collidepoint(event.pos):
79                      self.evaluate_guess("LOWER")
80
81          def update(self):
82              pass
83
84          def render(self, screen):
85              screen.fill((25, 80, 45))
86
```

```
87              title_surf = self.font_title.render("High-Low Card
Predictor", True, (245, 245, 245))
88              screen.blit(title_surf, (self.width // 2 -
title_surf.get_width() // 2, 25))
89
90              score_surf = self.font_medium.render(f"Score:
{self.score}", True, (255, 220, 80))
91              screen.blit(score_surf, (30, 30))
92
93              streak_color = (255, 150, 50) if self.streak >= 1 +
self.STREAK_STEP else (210, 210, 210)
94              streak_surf = self.font_small.render(f"Streak:
{self.streak}  x{self.multiplier}", True, streak_color)
95              screen.blit(streak_surf, (30, 60))
96
97              rem_surf = self.font_small.render(f"Deck:
{self.deck.remaining} left", True, (210, 210, 210))
98              screen.blit(rem_surf, (self.width -
rem_surf.get_width() - 30, 35))
99
100             card_w, card_h = 130, 180
101             self.current_card.render(screen, self.width // 2 -
card_w // 2, 100, card_w, card_h)
102
103             status_surf =
self.font_small.render(self.status_msg, True, self.status_color)
104             screen.blit(status_surf, (self.width // 2 -
status_surf.get_width() // 2, 310))
105
106             pygame.draw.rect(screen, (40, 140, 60),
self.btn_higher, border_radius=8)
107             pygame.draw.rect(screen, (220, 220, 220),
self.btn_higher, width=2, border_radius=8)
108             high_surf = self.font_medium.render("HIGHER", True,
(255, 255, 255))
109             screen.blit(
110                 high_surf,
111                 (self.btn_higher.centerx -
high_surf.get_width() // 2, self.btn_higher.centery -
high_surf.get_height() // 2),
112             )
113
114             pygame.draw.rect(screen, (170, 50, 50),
self.btn_lower, border_radius=8)
115             pygame.draw.rect(screen, (220, 220, 220),
self.btn_lower, width=2, border_radius=8)
116             low_surf = self.font_medium.render("LOWER", True,
(255, 255, 255))
117             screen.blit(
118                 low_surf,
119                 (self.btn_lower.centerx -
low_surf.get_width() // 2, self.btn_lower.centery -
low_surf.get_height() // 2),
120             )
```

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

[Tool result]
The file /Users/sonia/Software-Engineering-Lab/Lab4/55_card_predictor/game/game_engine.py has been updated successfully. (file state is current in your context — no need to Read it back)

User

```
[Tool result]
File created successfully at: /Users/sonia/Software-Engineering-Lab/
Lab4/55_card_predictor/tests/test_reveal.py (file state is current
in your context — no need to Read it back)
```

User

```
[Tool result]
test_edge_cases_9_10_face_cards
(tests.test_comparison.TestRankComparison.test_edge_cases_9_10_face_
cards) ... ok
test_equal_rank_is_push
(tests.test_comparison.TestRankComparison.test_equal_rank_is_push) .
.. ok
test_full_rank_order
(tests.test_comparison.TestRankComparison.test_full_rank_order) ...
ok
test_multiplier_schedule
(tests.test_streak.TestStreakMultiplier.test_multiplier_schedule) ..
. ok
test_push_at_zero_streak
(tests.test_streak.TestStreakMultiplier.test_push_at_zero_streak) ..
. ok
test_push_preserves_score_and_streak
(tests.test_streak.TestStreakMultiplier.test_push_preserves_score_an
d_streak) ... ok
test_render_shows_streak
(tests.test_streak.TestStreakMultiplier.test_render_shows_streak) ..
. ok
test_streak_builds_and_score_escalates
(tests.test_streak.TestStreakMultiplier.test_streak_builds_and_score
_escalates) ... ok
test_wrong_guess_resets_streak
(tests.test_streak.TestStreakMultiplier.test_wrong_guess_resets_stre
ak) ... ok
test_buttons_work_again_after_reveal
(tests.test_reveal.TestRevealPause.test_buttons_work_again_after_rev
eal) ... ok
test_clicks_ignored_during_reveal
(tests.test_reveal.TestRevealPause.test_clicks_ignored_during_reveal
) ... ok
test_guess_starts_reveal_with_both_cards
(tests.test_reveal.TestRevealPause.test_guess_starts_reveal_with_bot
h_cards) ... ok
test_push_and_wrong_also_reveal
(tests.test_reveal.TestRevealPause.test_push_and_wrong_also_reveal)
... ok
test_render_both_states
(tests.test_reveal.TestRevealPause.test_render_both_states) ... ok
test_reveal_lasts_for_pause_then_advances
(tests.test_reveal.TestRevealPause.test_reveal_lasts_for_pause_then_
```

advances) ... ok

Ran 15 tests in 1.133s

OK

Assistant

All 15 tests pass. I'll also render both screens to images and check the layout by eye:

User

[Tool result]
(Bash completed with no output)

Assistant

After each guess, the game now shows the previous card and the drawn card side by side for 1.2 seconds, then moves on to the next round. All 15 tests pass. I also rendered both screens to images and checked them.

How a round works now (all in [game_engine.py](game/game_engine.py)):
1. **Click HIGHER or LOWER.** The score and streak update straight away, using the same rules as before, including PUSH on a tie.
2. **Reveal.** The two cards sit side by side, labelled "Previous" and "Drawn" with "vs" between them, and the CORRECT, WRONG or PUSH message shows underneath. The buttons turn grey, and clicks during this time are ignored, so you can't draw extra cards or guess twice by accident.
3. **Next round.** After 1.2 seconds the drawn card moves to the centre as the new current card. The buttons get their colours back and the message returns to "Will the next card be HIGHER or LOWER?".

To change the pause length, edit `REVEAL_MS` at the top of the class. The timer runs in `update()`, which `main.py` already calls every frame, so I didn't need to change `main.py`.

Tests: the new tests are in tests/test_reveal.py. They check that:
- a guess shows both cards and updates the score straight away
- the reveal stays up for the full pause and then moves to the next round
- clicks during the reveal draw no extra cards and leave the score

alone
- the buttons work again after the reveal, and the streak carries on
- PUSH and WRONG results get the same reveal
- the screen draws without errors before, during and after the reveal

Run everything with:
```
.venv/bin/python -m unittest tests.test_comparison tests.test_streak tests.test_reveal -v
```